

BAB 2

PENGENALAN STRUKTUR DATA, *COMPREHENSION*, PERULANGAN DAN *SLICING*

2.1 TUJUAN PERCOBAAN

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Memahami berbagai struktur data yang tersedia dalam Python.
2. Mempelajari cara mengimplementasikan perulangan (looping) dalam Python.
3. Mengembangkan pemahaman dasar untuk menyusun logika pemrograman dalam Python.

2.2 ALAT-ALAT YANG DIGUNAKAN

Pada percobaan ini, akan digunakan berbagai alat dan bahan yang mendukung kegiatan praktikum. Alat-alat tersebut meliputi:

- Daftar alat utama: OS Windows/Linux/macOS, Anaconda, Google Colab

2.3 DASAR TEORI

Python adalah bahasa pemrograman tingkat tinggi yang banyak digunakan untuk berbagai kebutuhan seperti pengembangan web, analisis data, kecerdasan buatan, dan otomatisasi tugas. Python memiliki sintaks yang mudah dipahami dan mendukung paradigma pemrograman prosedural, berorientasi objek, serta fungsional.

2.3.1 STRUKTUR DATA DALAM PYTHON

Python memiliki beberapa struktur data utama yang digunakan untuk menyimpan dan mengelola data:

- *List*: Kumpulan elemen yang dapat berubah (mutable) dan dapat menyimpan berbagai tipe data.
- *Tuple*: Mirip dengan *list* tetapi bersifat immutable (tidak bisa diubah setelah dibuat).
- *Set*: Kumpulan elemen unik tanpa urutan tertentu yang digunakan untuk operasi himpunan.
- *Dictionary*: Koleksi pasangan kunci-nilai yang memungkinkan pengambilan data berdasarkan kunci tertentu.

2.3.2 PERULANGAN DALAM PYTHON

Perulangan atau looping atau iteration adalah pengulangan satu atau sejumlah pernyataan bila kondisi terpenuhi. Pengulangan bisa dilakukan sebanyak n kali atau tergantung suatu kondisi.

Python menyediakan dua jenis perulangan utama:

- *For Loop*: Digunakan untuk mengiterasi elemen dalam iterable seperti *list*, *tuple*, dan *dictionary*.
- *While Loop*: Digunakan untuk perulangan berbasis kondisi, dan loop akan terus berjalan selama kondisi tertentu terpenuhi.

2.3.2.1 Perulangan For

Perulangan *for* digunakan untuk menentukan perulangan sebanyak n kali (bukan pernyataan Boolean) Python mendukung beberapa model perulangan *for*. Sintaksis dasarnya adalah:

```
for i in range(5):  
    pernyataan
```

Penjelasan:

- Perulangan ini akan mengeksekusi pernyataan untuk setiap nilai dari `i` dalam urutan yang dihasilkan oleh `range(5)`, yaitu 0, 1, 2, 3, 4.
- pernyataan dijalankan sebanyak 5 kali, sekali untuk setiap nilai `i`.

Contoh:

```
for i in range(5):  
    print(f"Nomor {i}")
```

Kapan pernyataan dijalankan?

- `print(f"Nomor {i}")` dijalankan sebanyak 5 kali, dengan `i` berturut-turut memiliki nilai 0, 1, 2, 3, dan 4.

2.3.2.2 Perulangan While

Perulangan `while` membutuhkan kondisi yang bersifat Boolean. Apabila kondisi Boolean terpenuhi maka akan dijalankan pengulangan hingga kondisi tidak lagi terpenuhi. Sintaks dasarnya adalah:

```
while kondisi:  
    pernyataan
```

Penjelasan:

- Perulangan ini akan terus mengeksekusi pernyataan selama kondisi bernilai benar (`True`).
- Perulangan ini berhenti ketika kondisi menjadi salah (`False`).

Contoh:

```
i = 0  
  
while i < 5:  
    print(f"Nomor {i}")  
    i += 1 # Increment untuk menghentikan loop
```

Kapan pernyataan dijalankan?

`print(f"Nomor {i}")` dijalankan sebanyak 5 kali, selama nilai `i` lebih kecil dari 5. Nilai `i` akan bertambah 1 setiap kali perulangan, sehingga akhirnya kondisi `i < 5` menjadi `False` dan loop berhenti.

2.3.3 COMPREHENSION

Comprehension adalah cara yang lebih ringkas untuk membuat koleksi (list, dictionary, set) dengan menggunakan sintaks yang mirip dengan notasi matematika. Dengan *comprehension*, kode yang biasanya memerlukan beberapa baris dengan *loop* dan *conditional* bisa ditulis dalam satu baris.

Mengapa Menggunakan Comprehension?

Keringkasan: Mengurangi jumlah baris kode.

Keterbacaan: Jika digunakan dengan bijak, dapat membuat kode lebih mudah dipahami.

Efisiensi: Dalam banyak kasus, *comprehension* bisa lebih cepat daripada perulangan biasa.

2.3.3.1 List Comprehension

List comprehension adalah cara ringkas untuk membuat *list* baru dengan menerapkan ekspresi pada setiap elemen dari iterable.

Sintaks Dasar:

```
[ ekspresi for item in iterable if kondisi ]
```

2.3.3.2 Dictionary Comprehension

Dictionary comprehension adalah teknik pembuatan *dictionary* secara langsung dengan mengiterasi dan menentukan pasangan kunci-nilai.

Sintaks Dasar:

```
{ key_expression: value_expression for item in iterable if kondisi }
```

2.3.3.3 Set Comprehension

Set comprehension adalah metode untuk membuat set dengan cara yang mirip seperti *list* comprehension, namun hasilnya berupa set yang hanya berisi elemen unik.

Sintaks Dasar:

```
{ ekspresi for item in iterable if kondisi }
```

2.3.4 SLICING DATA

Slicing merupakan teknik untuk mengambil sebagian data (subsekuen) dari sebuah urutan (sequence) seperti *list*, tuple, atau string dengan menggunakan notasi indeks. Dengan slicing, kita dapat dengan mudah mengekstrak elemen yang diinginkan tanpa harus menulis loop secara eksplisit.

Sintaks dasar untuk slicing adalah:

```
data[start:stop:step]
```

- **start:** Indeks awal (inklusif) dari data yang ingin diambil. Jika tidak dituliskan, secara default akan mulai dari awal urutan.
- **stop:** Indeks akhir (eksklusif) hingga data diambil. Data pada indeks ini tidak termasuk dalam hasil slicing.
- **step:** Langkah atau interval antara elemen yang diambil. Jika tidak dituliskan, secara default nilainya adalah 1.

2.4 PROSEDUR PERCOBAAN

2.4.1 STRUKTUR DATA DALAM PYTHON

2.4.1.1 List

List adalah struktur data yang dapat menyimpan beberapa nilai dalam satu variabel. Elemen dalam *list* dapat diubah (mutable) dan bisa memiliki tipe data yang berbeda.

Contoh studi kasus yang akan Anda lakukan adalah Mengelola Data Mahasiswa. *List* akan digunakan untuk menyimpan daftar nama mahasiswa.

```
mahasiswa_list = ["Ali", "Budi", "Citra", "Dewi"]  
print("List Mahasiswa:", mahasiswa_list)
```

Berikut cara mengakses elemen dalam *list*.

```
print("Mahasiswa pertama:", mahasiswa_list[0])  
print("Mahasiswa terakhir:", mahasiswa_list[-1])
```

2.4.1.2 Tuple

Tuple adalah struktur data yang mirip dengan *list*, tetapi bersifat immutable. Artinya, setelah tuple dibuat, isinya tidak dapat diubah.

Pada studi kasus ini, Tuple akan digunakan untuk menyimpan data mahasiswa yang bersifat tetap (immutable).

```
data_mahasiswa = ("Ali", 21, "Teknik Informatika")  
print("Tuple Mahasiswa:", data_mahasiswa)
```

Berikut cara mengakses elemen dalam *list*.

```
print("Nama Mahasiswa:", data_mahasiswa[0])
```

2.4.1.3 Set

Set adalah struktur data yang menyimpan elemen unik tanpa duplikasi. Elemen dalam set tidak memiliki urutan tertentu. Pada studi kasus ini, Set akan digunakan untuk menyimpan kumpulan NIM unik.

```
nim_set = {101, 102, 103, 104, 101} # Duplikat akan diabaikan  
print("Set NIM Mahasiswa:", nim_set)
```

2.4.1.4 Dictionary

Dictionary adalah struktur data yang menyimpan data dalam format pasangan kunci-nilai. Setiap kunci harus unik dan digunakan untuk mengakses nilai terkait. *Dictionary* akan digunakan untuk menyimpan informasi mahasiswa dengan pasangan kunci-nilai.

```
mahasiswa_dict = {  
    "nim": 101,  
    "nama": "Ali",  
    "jurusan": "Teknik Informatika",  
    "usia": 21  
}  
print("Dictionary Mahasiswa:", mahasiswa_dict)
```

Berikut cara mengakses elemen dalam *list*.

```
print("Nama Mahasiswa:", mahasiswa_dict["nama"])  
print("Jurusan:", mahasiswa_dict["jurusan"])
```

2.4.2 PERULANGAN

1. Percobaan perulangan while

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
x = 1  
n = int(input("Masukkan nilai n: "))  
jumlah = 0  
  
while x <= n:  
    jumlah = jumlah + x  
    x = x + 1  
  
print("Jumlah " + str(n) + " bilangan pertama " + str(jumlah))
```

2. Percobaan perulangan for 1 (tanpa range)

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
angka = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
  
for x in angka:  
    print(x, end=" ",)
```

3. Percobaan perulangan for 1 (dengan range)

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
angka = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
  
for index in range(len(angka)):  
    print(angka[index], end=" ",)
```

2.4.3 COMPREHENSION

2.4.3.1 List Comprehension

1. Percobaan membuat *list* berisi kuadrat dari angka 1 sampai 10

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
kuadrat = [x**2 for x in range(1, 11)]  
print(kuadrat)
```

2. Percobaan membuat *list* berisi angka genap dari 1 sampai 20:

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
genap = [x for x in range(1, 21) if x % 2 == 0]  
print(genap)
```

2.4.3.2 Dictionary Comprehension

1. Percobaan membuat *dictionary* dan kuncinya adalah angka dari 1 sampai 5 dan nilainya adalah kuadratnya

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
dict_kuadrat = {x: x**2 for x in range(1, 6)}  
print(dict_kuadrat)
```

2. Percobaan membuat *dictionary* dengan hanya angka genap

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
dict_genap = {x: x**2 for x in range(1, 11) if x % 2 == 0}  
print(dict_genap)
```

2.4.3.3 Set Comprehension

1. Percobaan membuat set berisi kuadrat dari angka 1 sampai 10:

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
set_kuadrat = {x**2 for x in range(1, 11)}  
print(set_kuadrat)
```

2. Percobaan mendapatkan himpunan karakter unik dari sebuah string

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
kata = "komprehensif"  
unik = {huruf for huruf in kata}  
print(unik)
```

2.4.4 SLICING DATA

1. Percobaan menggunakan *slicing* pada *List*

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
angka = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
  
# Mengambil elemen dari indeks 2 sampai 5 (elemen pada indeks 5 tidak termasuk)  
sub_list = angka[2:5]  
print(sub_list) # Output: [2, 3, 4]  
  
# Mengambil elemen dari awal sampai indeks 4  
sub_list_awal = angka[:4]  
print(sub_list_awal) # Output: [0, 1, 2, 3]  
  
# Mengambil elemen dari indeks 5 sampai akhir  
sub_list_akhir = angka[5:]  
print(sub_list_akhir) # Output: [5, 6, 7, 8, 9]  
  
# Menggunakan step untuk mengambil setiap elemen kedua  
sub_list_step = angka[::2]  
print(sub_list_step) # Output: [0, 2, 4, 6, 8]
```

2. Percobaan menggunakan *slicing* pada *String*

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
teks = "PythonProgramming"  
  
# Mengambil substring dari indeks 6 sampai 16  
substring = teks[6:16]  
print(substring) # Output: Programming  
  
# Membalik string menggunakan slicing dengan step negatif  
teks_terbalik = teks[::-1]  
print(teks_terbalik) # Output: gnimmargorPnohtyP
```


3. Percobaan menggunakan *slicing* dengan nilai negatif

Ketikkan kode program di bawah ini, lalu jalankan melalui `Run` di *toolbar* atau ketik `Shift + Enter`. Tuliskan keluarannya!

```
angka = [10, 20, 30, 40, 50]

# Mengambil elemen terakhir dengan indeks -1
elemen_terakhir = angka[-1]
print(elemen_terakhir) # Output: 50

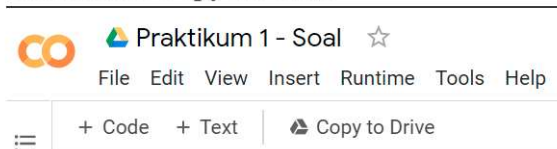
# Mengambil elemen dari indeks -4 sampai -1 (elemen pada indeks -1 tidak termasuk)
sub_list_negatif = angka[-4:-1]
print(sub_list_negatif) # Output: [20, 30, 40]
```

2.5 TUGAS

1. Buatlah program yang menggunakan *list*, *tuple*, *set*, dan *dictionary* untuk menyimpan data produk di sebuah toko online.
 - a. *List*: Menyimpan daftar nama produk.
 - b. *Tuple*: Menyimpan informasi produk (nama, harga, stok).
 - c. *Set*: Menyimpan ID unik produk.
 - d. *Dictionary*: Menyimpan informasi produk dengan kunci sebagai nama produk.
2. Buatlah program yang menggunakan *for loop* dan *while loop* untuk:
 - a. Menampilkan semua produk dalam daftar toko online
 - b. Mengurangi stok produk setiap kali terjadi pembelian sampai stok habis.
3. Buatlah *list* baru yang berisi nama produk dengan stok di atas nilai tertentu (misalnya, $\text{stok} > 20$) dari data di *dictionary*.
4. Gunakan *slicing* pada *list* nama produk untuk menampilkan sebagian daftar produk, misalnya 3 produk pertama.

Petunjuk pengerjaan soal:

1. Buka tautan berikut, pastikan Anda *login* menggunakan **akun Student UB**.
<https://s.ub.ac.id/nbkal02>
2. Klik tombol **Copy to Drive**



3. Beri nama file **Praktikum 2 - Nama - NIM.ipynb**
4. Isilah *cell* yang kosong.
5. Download file ***.ipynb** dengan cara klik **File > Download .ipynb**
6. Kumpulkan file ***.ipynb** ke asisten

2.6 KESIMPULAN

Pengenalan struktur data, *comprehension*, perulangan dan *slicing* dalam Python merupakan fondasi utama dalam pengembangan program yang efisien dan terstruktur. Pemahaman tentang berbagai jenis struktur data seperti *list*, *tuple*, *set*, dan *dictionary* memungkinkan pengembang untuk memilih metode penyimpanan data yang paling sesuai berdasarkan kebutuhan aplikasi. *List* memberikan fleksibilitas dalam memanipulasi data, *tuple* menjamin konsistensi data yang bersifat tetap, *set* memastikan keunikan elemen, dan *dictionary* mempermudah pencarian data berbasis kunci.

Penerapan perulangan menggunakan *for* dan *while* loop mempermudah proses iterasi dan pengolahan data dalam jumlah besar. Penggunaan teknik *comprehension* pada *list*, *set*, dan *dictionary* mempercepat proses pembuatan struktur data yang kompleks dengan sintaks yang lebih ringkas dan efisien.

Selain itu, pemahaman tentang teknik *slicing* memungkinkan pengolahan data secara spesifik dan selektif, mendukung manipulasi data yang lebih efektif. Dengan menguasai konsep-konsep dasar ini, pengembang dapat membangun algoritma yang lebih kompleks dan efisien dalam berbagai aplikasi pemrograman, mulai dari pengolahan data sederhana hingga pengembangan sistem berbasis data yang kompleks.